Lecture 9 — Bit Manipulation & Bitmask Enumeration

> Year Curriculum · Prerequisites: Week 2 (C++/STL), Week 3 (complete search), Week 8 (recursion & backtracking) · Estimated time: 2 × 90 min

1. Overview

Computers store integers in binary, and C++ exposes that representation through a handful of operators that run in a single CPU instruction. Competitive programmers use them in two ways:

1. **As tricks on numbers**: testing parity, finding the lowest set bit, counting ones, reasoning about XOR sums, solving AND/OR/XOR problems one bit at a time.
2. **As sets**: an `int` with n ≤ 31 bits *is* a subset of {0, …, n−1}. Union, intersection and membership are single instructions, and "for every subset" becomes a plain `for` loop over integers.

Last week we enumerated subsets recursively. This week we do it iteratively with bitmasks, learn to enumerate the submasks of a mask in $O(3^n)$ total, and collect the bit identities you will keep using: in Fenwick trees (Week 24, `x & -x`), bitmask DP (Week 26), Nim and Sprague–Grundy (Week 33, XOR), and XOR bases (Nationals track).

2. Binary representation

An unsigned integer x with bits b_{k−1} … b_1 b_0 has value $\Sigma\, b_i \cdot 2^i$. Bit 0 is the least significant bit (LSB). For example 44 = 32 + 8 + 4 = `101100`$_2$, with bits 2, 3 and 5 set.

C++ `int` is 32 bits and `long long` is 64 bits on every judge you will use. Signed integers use **two's complement**: the negative of x is `~x + 1` (flip all bits, add one). Consequences:

- `-1` is all ones; `~0 == -1`.
- The highest bit is the sign bit; `INT_MIN` is `1` followed by 31 zeros, and `-INT_MIN` overflows.
- `x & -x` isolates the lowest set bit (section 3.2).

Right-shifting a negative number was implementation-defined before C++20 (GCC does an arithmetic shift, copying the sign bit), and left-shifting into or past the sign bit is undefined. The practical rule: **do bit tricks on non-negative values**, and use `unsigned` or `long long` when you need bit 31 or higher.

3. Operators and identities

Expression	Meaning
`a & b`	AND: bit set in both
`a | b`	OR: bit set in either
`a ^ b`	XOR: bit set in exactly one
`~a`	NOT: flip every bit
`a << k`	shift left: $a \cdot 2^k$ (if no overflow)
`a >> k`	shift right: $\lfloor a / 2^k \rfloor$ for $a \geq 0$

Precedence trap. Comparison operators bind *tighter* than `&`, `^`, `|`. So `if (x & 1 == 0)` means `x & (1 == 0)`, i.e. `x & 0`, which is always false. Likewise `a ^ b < c` means `a ^ (b < c)`. Always parenthesise: `if ((x & 1) == 0)`. Shifts bind tighter than comparisons but looser than `+`: `1 << n - 1` is `1 << (n - 1)`.

3.1 Single-bit operations

```
bool on = (x >> i) & 1;    // test bit i
x |=  (1 << i);            // set bit i
x &= ~(1 << i);            // clear bit i
x ^=  (1 << i);            // toggle bit i
```

The literal `1` is an `int`. For bits 31 and above in a 64-bit variable you must write `1LL << i`; `1 << 40` is undefined behaviour (in practice it often gives 256 or 0), and this bug survives small tests and fails on large ones.

3.2 Lowest set bit

Subtracting 1 from x turns the lowest set bit into 0 and all the zeros below it into 1s:

```
x          = 0010 1100   (44)
x - 1      = 0010 1011
x & (x-1)  = 0010 1000   lowest set bit cleared
-x         = 1101 0100   (~x + 1, shown in 8 bits)
x & -x     = 0000 0100   lowest set bit isolated
```

Uses:

- `x & (x - 1) == 0` (for $x > 0$) $\Leftrightarrow$ x is a power of two.
- `for (int t = x; t; t &= t - 1)` visits each set bit once; `__builtin_ctz(t)` gives its index. This is O(popcount) instead of O(32).
- `x & -x` is the step size in a Fenwick tree (Week 24).

3.3 Builtins and `bitset`

GCC provides intrinsics that compile to single instructions:

Builtin	Returns	For 44
`__builtin_popcount(x)`	number of set bits	3
`__builtin_ctz(x)`	count trailing zeros = index of lowest set bit	2
`__builtin_clz(x)`	count leading zeros in 32 bits	26
`31 - __builtin_clz(x)`	index of highest set bit = $\lfloor \log_2 x \rfloor$	5

Append `ll` (`__builtin_popcountll`, `__builtin_ctzll`, `__builtin_clzll`) for 64-bit arguments, and then use `63 - clzll`. Passing a `long long` to the 32-bit version silently truncates it. `ctz(0)` and `clz(0)` are undefined. C++20 adds portable equivalents in `<bit>`, but many judges still compile as C++17.

`std::bitset<N>` is a fixed-size array of bits with `count()`, `test()`, `set()`, `reset()`, `flip()`, shifts and bitwise operators. Bulk operations process 64 bits per machine word, so they are about 64 times faster than the same loop over `bool`. A classic use is subset-sum reachability: `bitset<100001> can; can[0] = 1; for (int w : weights) can |= can << w;` runs in $O(n \cdot S / 64)$.

3.4 XOR identities

XOR is addition modulo 2 in each bit, so it is commutative, associative, `a ^ a = 0` and `a ^ 0 = a`. Hence:

- XOR-ing a list cancels pairs. If every value appears twice except one, the XOR of the whole list is that value.
- XOR has an inverse (itself), so **prefix XOR** works like prefix sums from Week 7: with `px[i] = a[1] ^ … ^ a[i]`, the XOR of `a[l..r]` is `px[r] ^ px[l-1]`.
- `a ^ b` has a 1 exactly where a and b differ.
- `a + b = (a ^ b) + 2 · (a & b)`: XOR is addition without carries, and AND marks where carries originate.

3.5 Thinking one bit at a time

AND, OR and XOR act on each bit independently. When a problem asks about a sum or count over such operations, split it by bit. Example: compute $\Sigma_{i<j} (a_i \text{ XOR } a_j)$ for $n = 10^5$. For a fixed bit b, a pair contributes 2^b exactly when the two numbers differ at bit b. If c_1 numbers have bit b set and $c_0 = n - c_1$ do not, there are $c_0 \cdot c_1$ such pairs. Summing over the 30 bits gives the answer in $O(30n)$ instead of $O(n^2)$. Worked check on `{1, 2, 3, 7}`: bit 0 is set in {1, 3, 7} ($c_1 = 3$, $c_0 = 1$) contributing $3 \cdot 1 \cdot 1 = 3$; bit 1 is set in {2, 3, 7} contributing $3 \cdot 1 \cdot 2 = 6$; bit 2 is set in {7} contributing $1 \cdot 3 \cdot 4 = 12$; total 21, which matches the brute force.

4. Subsets as bitmasks

Fix a universe {0, 1, …, n−1}. A subset S corresponds to the integer whose bit i is 1 exactly when i ∈ S.

Set operation	Bitmask expression
empty set, full set	`0`, `(1 << n) - 1`
i ∈ S	`(S >> i) & 1`
add i / remove i	`S | (1 << i)` / `S & ~(1 << i)`
S ∪ T, S ∩ T	`S | T`, `S & T`
S \ T	`S & ~T`
complement within the universe	`S ^ full`
|S|	`__builtin_popcount(S)`
S ⊆ T	`(S & T) == S`

4.1 Enumerating all subsets

Every integer from 0 to $2^n - 1$ is a distinct subset, so

```
for (int mask = 0; mask < (1 << n); mask++) { ... }
```

visits all of them. Worked example, CSES 1623 *Apple Division*: for each mask compute the weight s of the chosen group; the other group weighs total − s, and the difference is |total − 2s|. With weights `3 2 7 4 1` (total 17), the mask selecting {3, 2, 4} has s = 9 and difference |17 − 18| = 1, which is optimal. Checking each mask costs O(n), giving $O(n \cdot 2^n) \approx 2 \cdot 10^7$ for n = 20.

Two properties of the increasing order are worth remembering: a submask is numerically ≤ its supermask, so when you process masks in increasing order, all proper submasks of `mask` have already been processed. That is exactly the order bitmask DP needs (Week 26). And `mask & (mask - 1)` (drop lowest element) is smaller than `mask`, so you can compute any additive quantity incrementally:

```
sum[0] = 0;
for (int mask = 1; mask < (1 << n); mask++)
    sum[mask] = sum[mask & (mask - 1)] + p[__builtin_ctz(mask)];
```

This is $O(2^n)$ time but $O(2^n)$ memory: for n = 20, 2^{20} `long long`s is 8 MB, fine; for n = 27 it is 1 GB, not fine.

Bitmask vs. recursion. The bitmask loop is shorter and has no recursion overhead, but it cannot prune: it always visits all 2^n masks. If pruning matters (Week 8), recursion wins.

4.2 Enumerating submasks

Given a mask m, the loop

```
for (int s = m; s > 0; s = (s - 1) & m) { ... }
```

visits every non-empty submask of m in decreasing numeric order. Why: `s - 1` clears the lowest set bit of s and sets all bits below it; `& m` keeps only the positions that belong to m. The result is the largest submask of m that is smaller than s. For m = `1011` the sequence is `1011, 1010, 1001, 1000, 0011, 0010, 0001`. The empty submask must be handled separately, or use the form `for (s = m; ; s = (s - 1) & m) { ...; if (s == 0) break; }`.

A mask with k bits has 2^k submasks, so iterating over all masks and all their submasks costs $\Sigma_m 2^{\text{popcount}(m)} = \Sigma_k C(n, k)\, 2^k = \mathbf{3^n}$ (binomial theorem with $(1 + 2)^n$). Intuitively, each element is in one of three states: outside m, inside m but outside s, or inside s. For n = 10 the double loop runs exactly 59,049 = 3^{10} times. Budget: n ≤ 15–16 is safe ($3^{15} \approx 1.4 \cdot 10^7$), n = 20 is $3.5 \cdot 10^9$ and too slow.

4.3 Gray code

The reflected Gray code orders all n-bit strings so that consecutive strings differ in exactly one bit. The i-th code is `i ^ (i >> 1)`. For n = 3: `000 001 011 010 110 111 101 100`. CSES 2205 asks you to print it. Beyond the puzzle, Gray order lets you walk all subsets while adding or removing a single element at each step: if maintaining your quantity under one insertion or deletion is O(1), the whole walk is $O(2^n)$ without the $O(2^n)$ memory of the incremental-sum table. The bit that changes between step $i - 1$ and step i is `__builtin_ctz(i)`.

4.4 Other uses of bitmasks as state

- **Letters in a word**: a 26-bit mask per word. Two words share no letter iff `(m1 & m2) == 0` — an O(1) test instead of O(length).
- **Parity vectors**: keep a 10-bit mask whose bit d is the parity of the count of digit d in a prefix. A substring can be permuted into a palindrome iff the XOR of its two prefix masks has at most one bit set. With a hash map of mask counts this counts such substrings in O(n · 10).
- **Rows of a grid**: a row of ≤ 20 cells is an int; `row << 1` and `row >> 1` give the neighbours, and `(a & b) == 0` checks two rows for conflicts. This becomes broken-profile DP later in the course.
- **Visited sets** in small graphs (≤ 20 vertices): the foundation of TSP DP in Week 26.

5. Reference implementations

5.1 CSES 1623 — Apple Division

```cpp
#include <bits/stdc++.h>
using namespace std;
// CSES 1623 - Apple Division: split n <= 20 weights into two groups,
// minimise the difference of the group sums.
int main() {
    int n; cin >> n;
    vector<long long> p(n);
    long long total = 0;
    for (auto &x : p) { cin >> x; total += x; }
    long long best = LLONG_MAX;
    for (int mask = 0; mask < (1 << n); mask++) {
        long long s = 0;
        for (int i = 0; i < n; i++)
            if (mask >> i & 1) s += p[i];
        best = min(best, llabs(total - 2 * s));
    }
    cout << best << "\n";
}
```

(`mask >> i & 1` is safe without extra parentheses because `>>` binds tighter than `&`; when in doubt, add them.)

5.2 Bit tricks demo

```cpp
#include <bits/stdc++.h>
using namespace std;

void printBits(int x, int w) {
    for (int i = w - 1; i >= 0; i--) cout << (x >> i & 1);
}

int main() {
    int x = 0b101100;  // 44
    cout << "x           = "; printBits(x, 8); cout << " (" << x << ")\n";
    cout << "x & -x      = "; printBits(x & -x, 8); cout << "  lowest set bit\n";
    cout << "x & (x-1)   = "; printBits(x & (x - 1), 8); cout << "  clear lowest set
bit\n";
    cout << "popcount    = " << __builtin_popcount(x) << "\n";
    cout << "ctz         = " << __builtin_ctz(x) << "\n";
    cout << "31 - clz    = " << 31 - __builtin_clz(x) << "  (index of highest set
bit)\n";

    // Enumerate all submasks of m (non-empty), in decreasing order.
    int m = 0b1011;
    cout << "submasks of 1011:";
    for (int s = m; s > 0; s = (s - 1) & m) { cout << ' '; printBits(s, 4); }
```

```
    cout << " 0000\n";

    // Iterate over the set bits of m.
    cout << "set bits of 1011:";
    for (int t = m; t; t &= t - 1) cout << ' ' << __builtin_ctz(t);
    cout << "\n";

    // Sum over all masks of 3^n check: total submask iterations for n = 10.
    int n = 10; long long iters = 0;
    for (int mask = 0; mask < (1 << n); mask++)
        for (int s = mask; ; s = (s - 1) & mask) { iters++; if (s == 0) break; }
    cout << "total submask iterations for n=10: " << iters << " (3^10 = 59049)\n";
}
```

5.3 CSES 2205 — Gray Code

```
#include <bits/stdc++.h>
using namespace std;
// CSES 2205 - Gray Code: print all 2^n bit strings, consecutive ones differ in one bit.
int main() {
    int n; cin >> n;
    string out;
    for (int i = 0; i < (1 << n); i++) {
        int g = i ^ (i >> 1);
        for (int b = n - 1; b >= 0; b--) out += char('0' + (g >> b & 1));
        out += '\n';
    }
    cout << out;
}
```

5.4 Sum of pairwise XOR, bit by bit

```
#include <bits/stdc++.h>
using namespace std;
// Sum of (a[i] XOR a[j]) over all pairs i < j, with a[i] < 2^30 and n <= 1e5.
// Each bit is handled independently: O(n * 30).
int main() {
    int n; cin >> n;
    vector<long long> a(n);
    for (auto &x : a) cin >> x;
    long long total = 0;                        // <= (n^2/4) * 2^30 < 2^63 for n <= 1e5
    for (int b = 0; b < 30; b++) {
        long long ones = 0;
        for (auto x : a) ones += (x >> b) & 1;
        long long zeros = n - ones;
        total += ones * zeros * (1LL << b);   // pairs differing at bit b
    }
    cout << total << "\n";
}
```

Note the overflow analysis in the comment: for $n = 2 \cdot 10^5$ the true answer can exceed 2^{63} and you would need `__int128` or a modulus.

6. Common mistakes

- **Precedence**: `x & 1 == 0`, `a ^ b == c`, `mask & (1 << i) != 0` all parse wrongly. Parenthesise every bitwise sub-expression used in a comparison.
- **Shift width**: `1 << i` is an `int`. Use `1LL << i` for $i \geq 31$; shifting by $\geq$ the type width is undefined, even for `long long` ($i \geq 64$).
- **Wrong builtin width**: `__builtin_popcount(x)` with `long long x` drops the upper 32 bits; use `__builtin_popcountll`.
- **Builtins on zero**: `__builtin_ctz(0)` and `__builtin_clz(0)` are undefined. Guard or handle 0 specially.
- **Off-by-one in the loop bound**: `mask <= (1 << n)` visits a non-existent extra mask; the bound is `mask < (1 << n)`.
- **Forgetting the empty submask** in the `s = (s - 1) & m` loop, or writing `for (s = m; s >= 0; ...)`, which loops forever because `(0 - 1) & m == m`.
- **Memory blow-up**: tables indexed by mask need 2^n entries; 2^{25} ints is 128 MB.
- **Signed shifts**: right-shifting a negative value or left-shifting into the sign bit — keep values non-negative or unsigned.

7. Practice problems

Warm-up:

1. **Codeforces 579A — Raising Bacteria** — https://codeforces.com/problemset/problem/579/A (answer is a popcount).
2. **CSES 2205 — Gray Code** — https://cses.fi/problemset/task/2205
3. **CSES 1623 — Apple Division** — https://cses.fi/problemset/task/1623
4. **Codeforces 1097B — Petr and a Combination Lock** — https://codeforces.com/problemset/problem/1097/B (2^n sign choices).

Core:

5. **Codeforces 550B — Preparing Olympiad** — https://codeforces.com/problemset/problem/550/B
6. **AtCoder ABC 147 C — HonestOrUnkind2** — https://atcoder.jp/contests/abc147/tasks/abc147_c (guess who is honest as a bitmask, verify).
7. **AtCoder ABC 045 C — Many Formulas** — https://atcoder.jp/contests/abc045/tasks/arc061_a (choose where to put `+` signs).

8. **Codeforces 1420B — Rock and Lever** — https://codeforces.com/problemset/problem/1420/B (group by highest set bit).
9. **Codeforces 1615B — And It's Non-Zero** — https://codeforces.com/problemset/problem/1615/B (per-bit prefix counts).

Challenge:

10. **CSES 1146 — Counting Bits** — https://cses.fi/problemset/task/1146 (total ones in 1..n for $n \le 10^{15}$, per-bit counting).
11. **LeetCode 1371 — Find the Longest Substring Containing Vowels in Even Counts** — https://leetcode.com/problems/find-the-longest-substring-containing-vowels-in-even-counts/ (parity mask + prefix XOR).

8. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 10 "Bit manipulation" (bit representation, set representation, bit optimizations, DP over subsets): https://cses.fi/book/book.pdf
- CP-Algorithms, "Bit manipulation": https://cp-algorithms.com/algebra/bit-manipulation.html and "Enumerating submasks of a bitmask": https://cp-algorithms.com/algebra/all-submasks.html
- CP-Algorithms, "Gray code": https://cp-algorithms.com/algebra/gray-code.html
- Sean Eron Anderson, "Bit Twiddling Hacks": https://graphics.stanford.edu/~seander/bithacks.html
- Codeforces blog "C++ tips and tricks" and the GCC documentation of `__builtin_*` functions.
- Coming up: Week 24 (Fenwick trees use `x & -x`), Week 26 (bitmask DP), Week 33 (XOR in Nim).